

RHYNK

"Every Chat Has a Rhythm."

Low-Level Design (LLD)

Type: Implementation-level design — schemas, contracts, algorithms, sequence diagrams

Scope: Folder structure, WebSocket contracts, the three hardest problems, key flows

Companion: Database Design PDF (full ER diagrams), API Reference (full REST contracts), HLD document

Date: July 2026

Table of Contents

- 1 Codebase structure (backend + frontend)
- 2 The three hardest problems in this system
- 3 Sequence — message send flow
- 4 Sequence — music room join & NTP-style sync
- 5 Sequence — refresh token rotation under concurrency
- 6 Music room state machine
- 7 WebSocket event contracts (summary)
- 8 Where the rest of the detail lives

1. Codebase structure

Each backend module owns four files by convention — `routes.js`, `service.js`, `schema.js` (AJV validation), and a model file where relevant. Frontend is feature-first, mirroring the backend module split 1:1 so a given feature's client and server code are easy to reason about together.

1.1 Backend — `server/`

```
server/
├─ prisma/                schema.prisma + migrations
├─ src/
│  ├─ app.js              Fastify instance, autoloaded plugins
│  ├─ server.js           entry point
│  ├─ config/             env validation, constants
│  ├─ plugins/            prisma · mongoose · redis · jwt · rate-limit · socket · error-handler
│  └─ modules/
│     ├─ auth/            routes, service, schema, token utils
│     ├─ users/           + contacts, blocks
│     ├─ devices/         FCM token registration
│     ├─ conversations/   + membership
│     ├─ messages/        + Mongoose model, message:* socket handlers
│     ├─ music-rooms/     + sync engine, room:* socket handlers, JioSaavn client
│     ├─ songs/           + playlists, liked-songs
│     ├─ stories/         + Mongoose model (TTL index)
│     ├─ calls/           + call:* signaling relay
│     ├─ notifications/   + preferences, FCM client
│     └─ media/           R2 upload, FFmpeg transcode job
├─ jobs/                 scheduled cleanup, anonymization
├─ lib/                  firebase, logger, pagination helper
├─ sockets/              socket-auth middleware
└─ tests/                unit/ + integration/
```

1.2 Frontend — `client/`

```
client/
├─ src/
│  ├─ app/               socket.js · api-client.js · store/ (auth, presence, notifications)
│  └─ features/          one folder per backend module:
│                         auth, conversations, messages, music-room,
│                         songs, stories, calls, notifications, profile
│                         – each with components/, hooks/, api.js
│  └─ components/        shared dumb UI primitives
│  └─ layouts/           AppLayout, AuthLayout
│  └─ pages/             route-level containers
│  └─ styles/            Tailwind entrypoint
└─ vite.config.js / tailwind.config.js
```

2. The three hardest problems in this system

Rank	Problem	Why it's hard	Mitigation
1	Music room sync accuracy	Clock skew between clients causes desync. Poor networks cause variable round-trip time. DJ state changes race with sync pings.	NTP-style offset calculation (§4). Re-sync every 30s. Atomic Redis state reads. Visual sync indicator + manual re-sync button as a user-facing escape hatch.
2	Refresh token rotation under concurrency	Two simultaneous requests presenting the same refresh token must not both succeed — naive implementations allow token reuse, which is a security hole.	Redis <code>SETNX</code> (set-if-not-exists) to atomically claim a token before rotating. If the claim fails or the token doesn't match the current one on file, treat it as a stolen/replayed token and revoke the entire session family (§5).
3	Message ordering with cursor pagination	Getting this wrong shows messages out of order or duplicated when loading history. The last visible <code>_id</code> alone is not a reliable cursor if two messages share a timestamp.	Compound cursor: <code>{ created_at, _id }</code> . Sort on <code>created_at DESC</code> , break ties by <code>_id DESC</code> . Client sends its last visible cursor; server returns the next page strictly after it.

These three are ranked by difficulty, not by build order. Build order is dictated by dependencies (see HLD §7) — but expect these three specifically to need the most careful testing and the most willingness to revisit the first implementation.

3. Sequence — message send flow

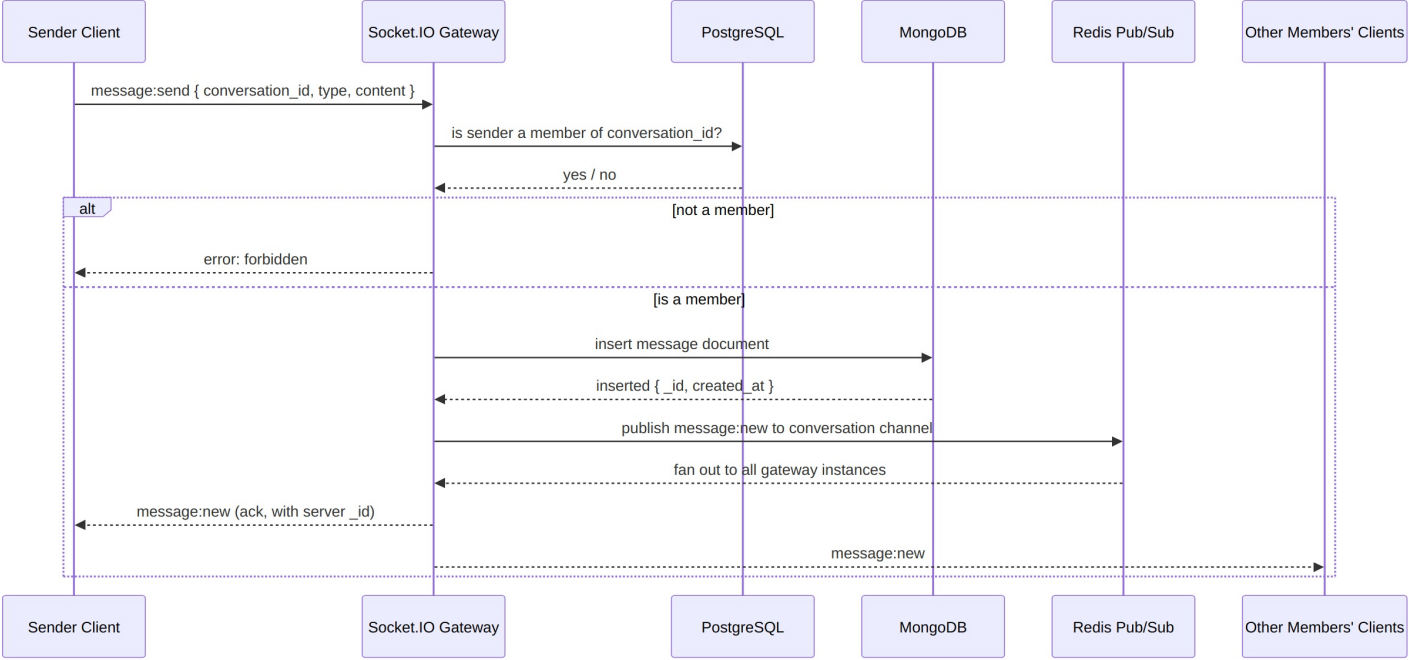


Figure 2 — a message send always re-checks membership server-side, even though the client already believes it's in the conversation

The membership check against Postgres on every single send is deliberate, not an oversight to optimize away — it's the one authorization check that closes the gap between "this socket is authenticated" and "this socket is allowed to write to this specific conversation." Skipping it (e.g. trusting a client-side conversation ID) would let a removed member keep sending messages until they reconnect.

4. Sequence — music room join & NTP-style sync

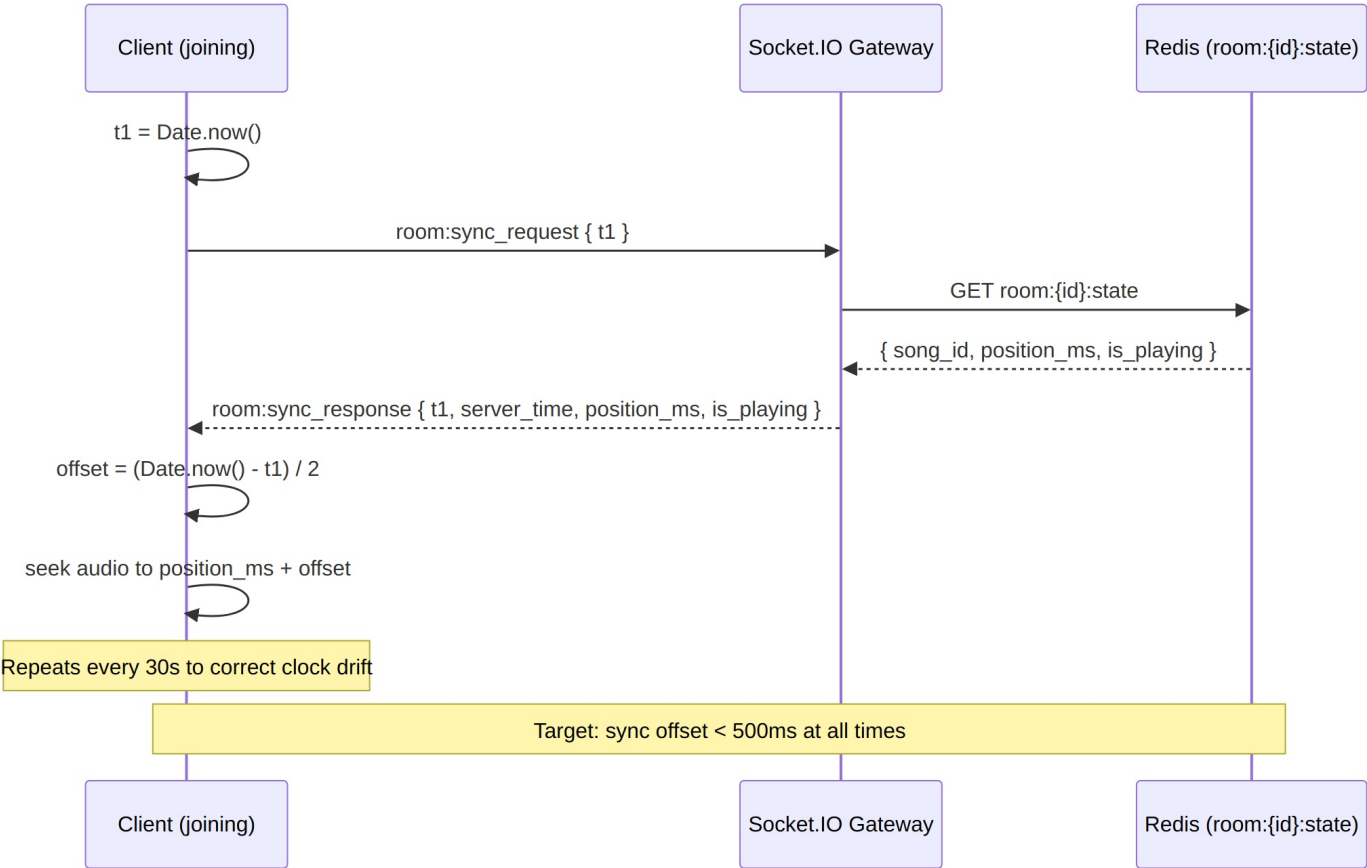


Figure 3 — the offset calculation compensates for one-way network latency by halving the observed round-trip

`offset = (Date.now() - t1) / 2` assumes the request and response legs of the round trip take roughly equal time — true enough in practice for the <500ms target, and simple enough to compute on every client without a heavier clock-sync protocol. Redis is the single source of truth for `position_ms` during a live session — it is never derived from PostgreSQL while a room is active; Postgres only receives a snapshot when the room ends (see state machine, §6).

5. Sequence — refresh token rotation under concurrency

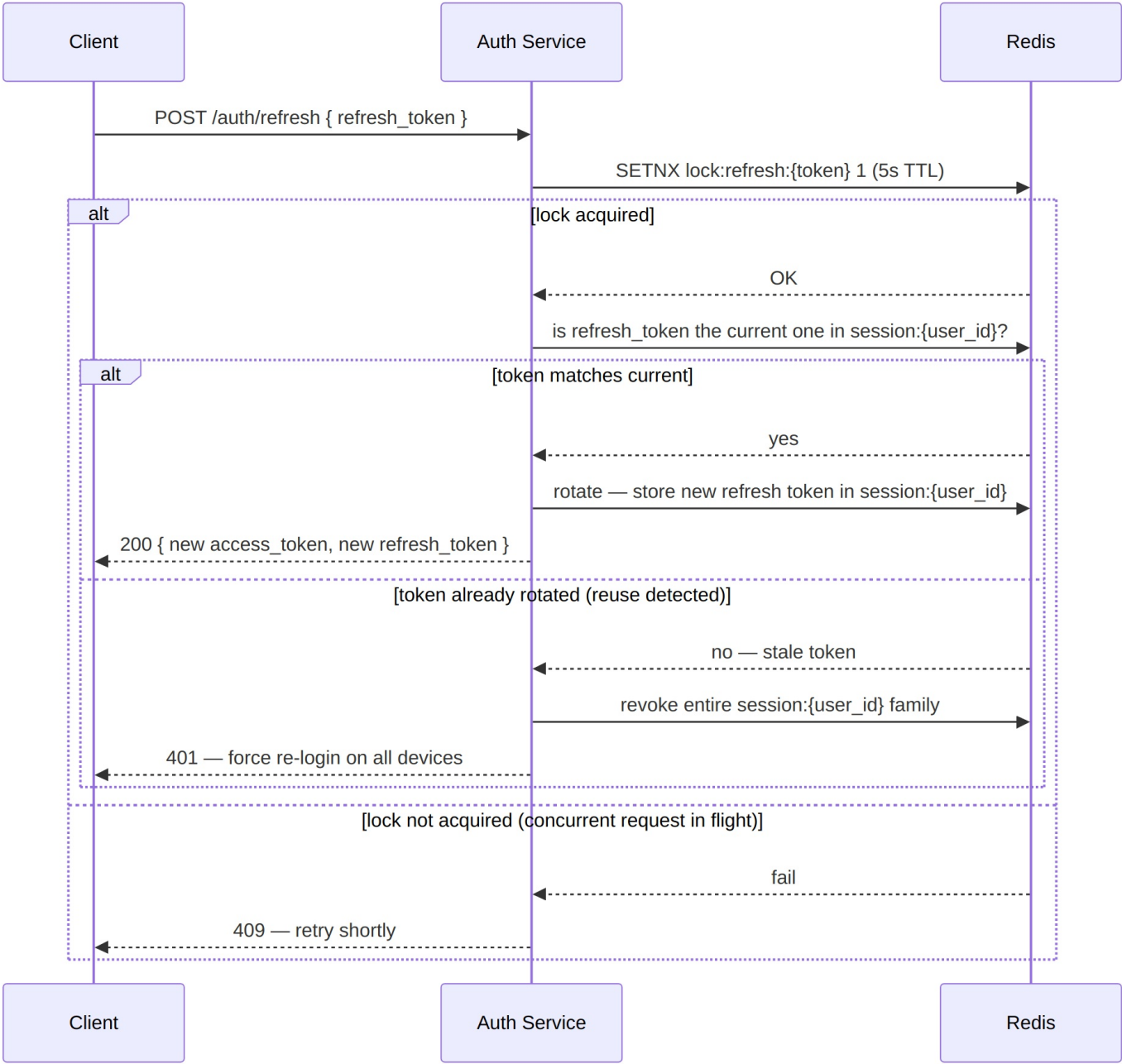


Figure 4 — SETNX as a distributed lock closes the race between two simultaneous refresh attempts

The failure mode this prevents: a stolen refresh token used by an attacker at the same moment the legitimate client also refreshes. Without the lock, both could succeed and mint valid new token pairs, meaning the theft goes undetected. With it, only one request wins the `SETNX`; if the token presented doesn't match what's on file as current (because it was already rotated a moment earlier), that's treated as reuse and the whole session family — every device — is revoked, forcing re-login everywhere.

6. Music room state machine

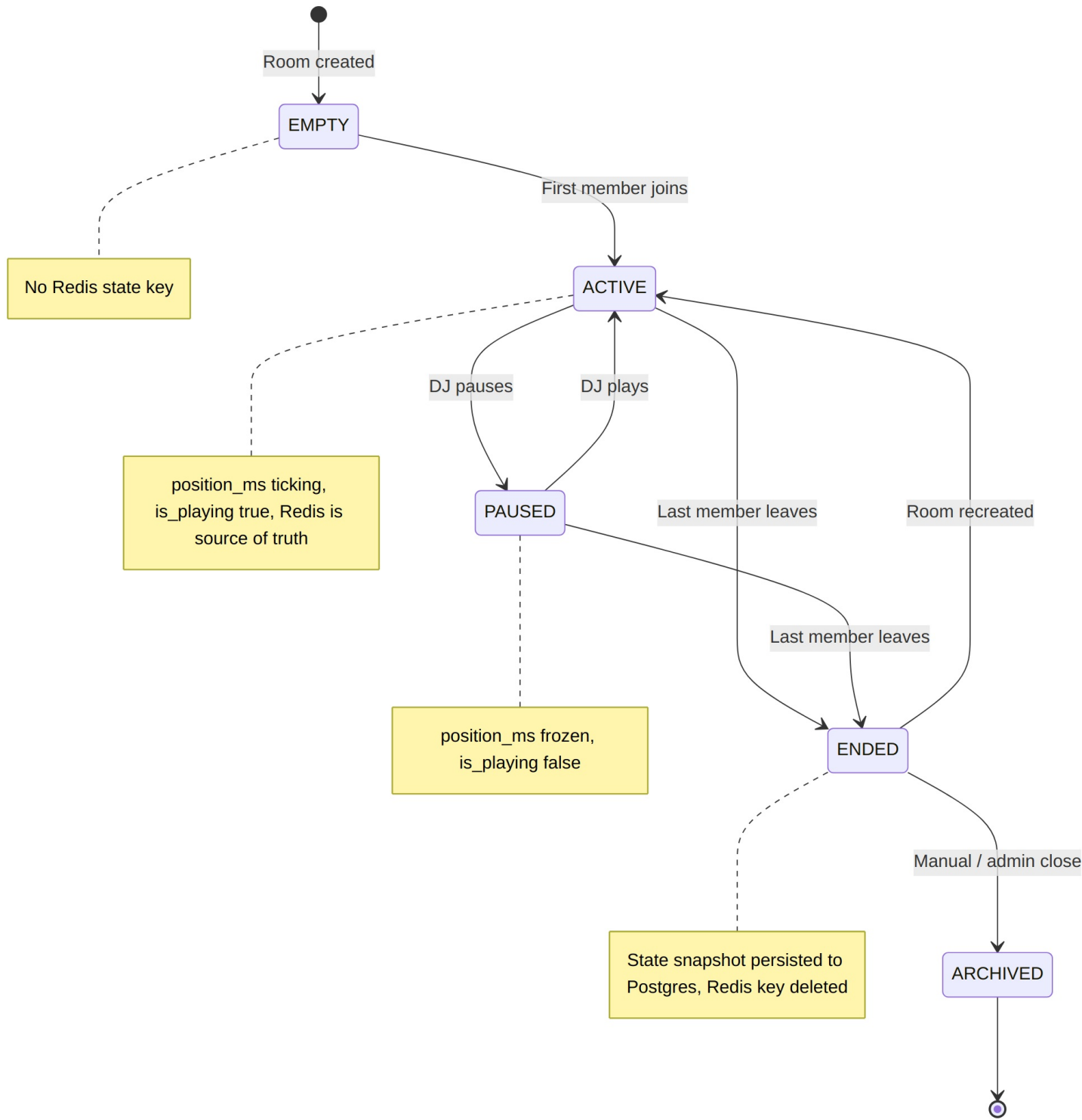


Figure 5 — EMPTY and ENDED have no Redis key; ACTIVE/PAUSED are the only states where Redis holds live playback data

The important asymmetry: a room can cycle between **ENDED** and **ACTIVE** indefinitely (people rejoin, someone plays a song again) — but **ARCHIVED** is a one-way exit, reachable only from a manual/admin close action, and represents the room being retired for good.

7. WebSocket event contracts (summary)

Full payload shapes for every event are in the **API Reference** document (§11–13). Summarized here for orientation:

Namespace	Key events	Auth model
Messaging	message:send/new/edit/delete/read/react , typing:* , presence:update	JWT verified once on socket handshake; membership re-checked per-event
Music Room	room:join , room:state , room:sync_request/response , room:play/pause/seek/skip , room:queue_add/remove , room:dj_changed	DJ-only events (play/pause/seek/skip/queue_remove) rejected server-side for non-DJ sockets, not just hidden client-side
Calls	call:initiate/incoming/accept/decline/offer/answer/ice_candidate/end/ended	Server only relays SDP/ICE between peers — never decodes or stores media

8. Where the rest of the detail lives

Document	Contains
Database Design PDF	Full PostgreSQL/MongoDB/Redis schemas, ER diagrams, indexing rationale, GDPR notes, scaling path
API Reference (markdown)	Every REST endpoint's method, auth, request/response body, error codes, and every WebSocket event's full payload
This LLD	The algorithms and flows that don't fit neatly into a schema or endpoint list — the "how" behind the three hardest problems
HLD (companion)	The big picture — services, data flow, tech stack, build roadmap